

A DSL Proposal for Deployment of Hybrid (Quantum-Classical) Service-Based Applications

Álvaro Manuel Aparicio

Morales

ITIS Software, Universidad de Málaga
Málaga, Andalucía, Spain
alvaroapamor@uma.es

Javier Troya

ITIS Software, Universidad de Málaga
Málaga, Andalucía, Spain
jtroya@uma.es

Jose García-Alonso

Quercus SEG

Universidad de Extremadura
Cáceres, Extremadura, Spain
jgaralo@unex.es

Carlos Canal

ITIS Software, Universidad de Málaga
Málaga, Andalucía, Spain
carloscanal@uma.es

Ernesto Pimentel

ITIS Software, Universidad de Málaga
Málaga, Andalucía, Spain
epimentel@uma.es

Juan Manuel Murillo

Quercus SEG

Universidad de Extremadura
Cáceres, Extremadura, Spain
juanmamu@unex.es

Abstract

Quantum computing promises to offer significant computational advantages for certain types of problems, which motivates its integration into current software systems. In this context, Service-Oriented Computing provides a suitable mechanism for encapsulating classical and quantum resources through the concept of hybrid quantum-classical services. Although techniques aimed at obtaining optimal deployments for this type of application are beginning to emerge, the literature still highlights the need for methodologies and tools to describe, validate, and homogenize the deployment structures of hybrid applications. To address this gap, this work proposes a MDE-based approach through the definition of a DSL for specifying deployment schemes for hybrid quantum-classical service applications. The proposal enables the description of the available classical and quantum resources, the services that compose the application, and the assignment of quantum services to QPUs, including the mapping between logical and physical qubits. In addition, a model-to-text transformation is provided to generate YAML artifacts that can be interpreted by deployment processes. Finally, the proposed grammar is instantiated on a hybrid search scenario to illustrate its applicability.

CCS Concepts

• **Software and its engineering** → **Process validation; Model-driven software engineering; Software implementation planning**; • **Hardware** → **Quantum computation**.

Keywords

Quantum Software Engineering, Model-driven Engineering, Domain-Specific Language, Deployment

ACM Reference Format:

Álvaro Manuel Aparicio Morales, Javier Troya, Jose García-Alonso, Carlos Canal, Ernesto Pimentel, and Juan Manuel Murillo. 2026. A DSL Proposal



This work is licensed under a Creative Commons Attribution 4.0 International License.
MODELS Companion 2026, Málaga, Spain
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2903-4/26/10
<https://doi.org/10.1145/3837062.3839074>

for Deployment of Hybrid (Quantum-Classical) Service-Based Applications. In *ACM/IEEE 29th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion 2026)*, October 04–09, 2026, Málaga, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3837062.3839074>

1 Introduction

Quantum computing is an emerging technology that promises significant computational advantages for certain types of problems [6]. This computational potential is based on the use of quantum - mechanical principles to perform computation. Therefore, it seems interesting to incorporate this new type of computer into future information systems. The union of both kinds of computing machines is known in the literature as hybrid quantum-classical systems.

However, as current software systems are mostly classical, it is necessary to explore how quantum resources can be integrated into these systems, leading to hybrid quantum-classical software systems. One approach to enabling this seamless integration is through Service-Oriented Computing (SOC). As proposed in the work of García-Alonso et al. [8], the conceptualization of hybrid services is one of the most appropriate approaches, given the current state of the Software Development Kits (SDKs) provided by different quantum cloud providers, which do not allow services to be developed in the same way as in classical computing [3].

Given the context of this type of applications, it becomes necessary to determine how they can be brought into production environments. This requires identifying, across both classical and quantum infrastructure providers, the set of computing resources on which the different services can be executed [5]. This challenge is known in the literature as *Placement Problem* [2]. Although this problem has been widely studied in classical service-based applications, the emergence of this new computational paradigm calls for the creation or adaptation of existing techniques to take into account for the specific characteristics of hybrid quantum-classical applications [6].

Tools and frameworks for obtaining deployment configurations for hybrid quantum-classical applications are starting to emerge [12]. Nevertheless, due to the current state of quantum computing in the Noisy Intermediate-Scale Quantum (NISQ) era [5], deploying

a quantum software component should not only focus on selecting a suitable QPU on which to execute the quantum algorithm. In addition, in order to progress towards fault-tolerant quantum computing, it is also necessary to consider the physical qubits onto which the circuit will be mapped.

In this regard, techniques that use QPU calibration data to guide the transpilation process and reduce the impact of noise are commonly referred to as noise-aware qubit layout selection or noise-aware initial layout selection [4]. Many of these techniques are commonly implemented within the IBM ecosystem, since it provides a wide range of quantum devices, exposes detailed calibration information, and regularly updates its SDK, named as Qiskit¹. However, given the existence and emergence of other quantum computing providers, relying on provider-specific mechanisms may limit the portability and interoperability of hybrid quantum-classical deployments. This calls for a higher-level modeling abstraction to capture deployment decisions.

From a Model-Driven Engineering (MDE) perspective, this motivates the need to provide a provider-independent specification mechanism for describing the deployment decisions of hybrid service-based applications at an appropriate level of abstraction. Such a mechanism should specify not only the computing nodes on which the different services are deployed, but also, in the case of quantum services, the physical qubits of the selected quantum device that will be used to execute the corresponding quantum circuit. This fine-grained deployment information is especially relevant in the current NISQ context where the quality of a quantum execution may depend not only on the selected QPU, but also on the specific physical qubits used, their connectivity, and their calibration data.

To address this objective, this work presents an initial proposal for a Domain-Specific Language (DSL) aimed at describing deployment configurations for hybrid quantum-classical service-based applications. The proposed (*HDML*), is defined according to a meta-model specifically tailored to hybrid quantum-classical deployments. It provides a structured and provider-independent way to specify the allocation of classical and quantum services to computing resources, as well as the mapping between logical qubits and physical qubits within a selected quantum backend. In this sense, the DSL can serve as a modeling artifact to make explicit deployment decisions that are usually hidden or delegated to provider-specific transpilation tools. Additionally, a model-to-text (M2T) transformation is provided in order to obtain a YAML file that serializes the deployment information into a de facto standard format used in classical deployment environments, such as Docker and Kubernetes.

The remainder of this paper is structured as follows. Section II presents the background. Section III introduces the proposed DSL for describing deployments of hybrid quantum-classical applications. Section IV discusses the related work in this area. Finally, Section V presents the conclusions and outlines future work.

2 Background

Modern service-based applications are usually required to satisfy Quality of Service (QoS) constraints, commonly established by clients or derived from system requirements. These constraints

¹<https://www.ibm.com/quantum/qiskit>

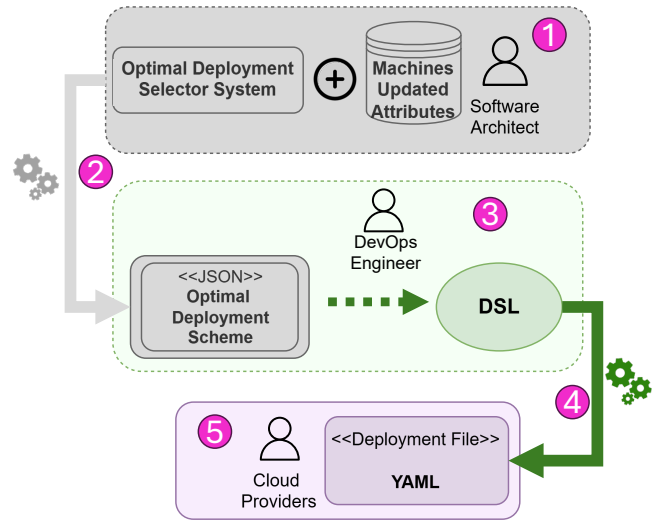


Figure 1: Overview of the framework to build deployments schemes of hybrid quantum-classical service applications

define acceptable thresholds for properties such as execution time, reliability, economic cost, or resource availability. With the emergence of hybrid quantum-classical service applications, these concerns become more complex, since deployment decisions must consider not only classical services, but also quantum services and the characteristics of the quantum resources on which they are executed [5].

This work builds on previous efforts aimed at obtaining optimal deployments for hybrid quantum-classical service-based applications [12]. Such work analyses trade-offs in the design space to identify the deployment that best satisfies user preferences. This is achieved by applying a utility function over the expected behaviour of the application on different computational resources, considering four dimensions: economic cost, execution time, reliability, and quantum error.

Once an optimal deployment has been obtained, the natural next step is to describe it in a structured way so that it can be interpreted and eventually enacted in real deployment environments. However, as pointed out by Garcia-Alonso et al. [5], hybrid quantum-classical deployments still lack homogeneous specification mechanisms. The present work addresses this need by proposing a DSL for defining deployment schemes in a homogeneous and provider-independent manner.

3 A DSL for Hybrid Applications Deployments

In this section, we first present an overview of the proposed approach for defining deployment schemes of hybrid service applications (Section 3.1). Then, we describe the domain-specific language (DSL) developed through its metamodel (Section 3.2), and apply it to an illustrative example (Section 3.3).

3.1 Overview

Figure 1 presents an overview of the framework within which the approach presented in this paper is framed. Its aim is to generate

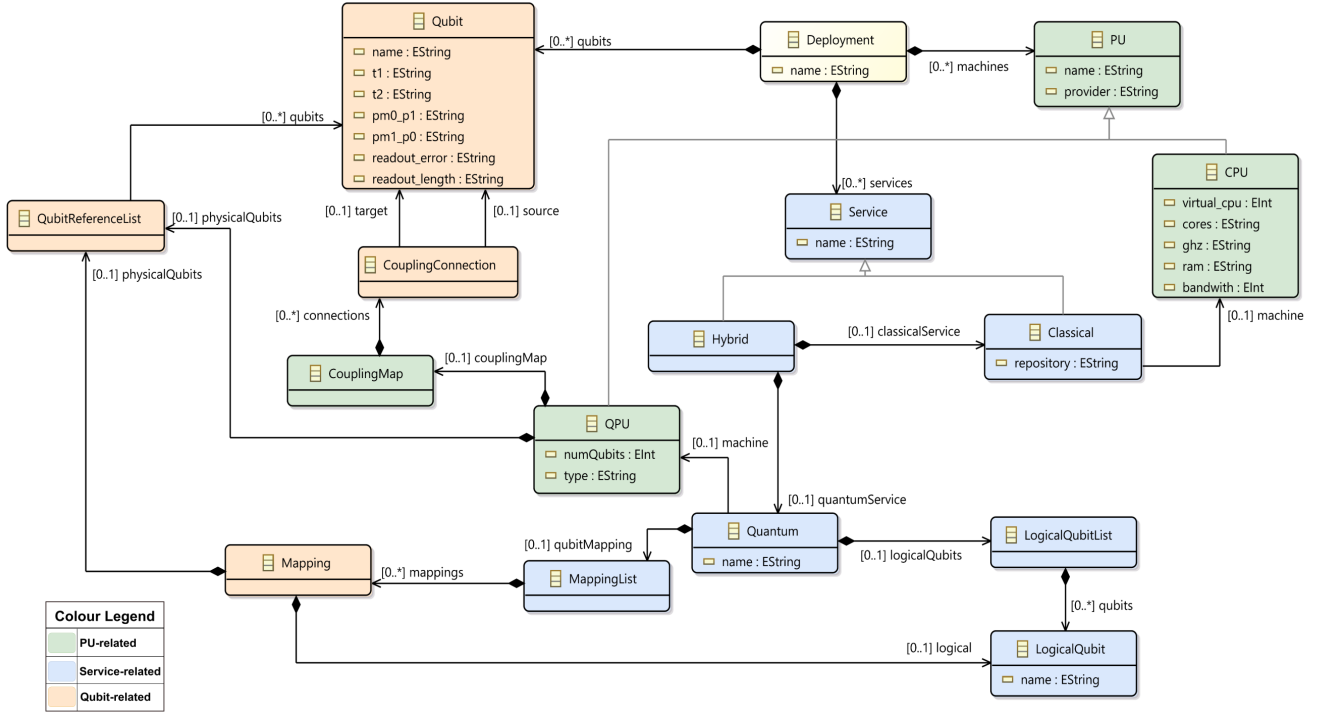


Figure 2: Metamodel of Hybrid Deployments Modeling Language (HDML)

deployment artifacts for applications distributed across different classical and quantum computing providers. Parts depicted in gray represent previous work of the authors [12], while green and purple parts display the contribution of this paper.

Starting with the top-most part of the figure, our framework supports software architects in making architectural decisions according to customer preferences ①. To this end, it enables the generation of a set of candidate deployment configurations based on real-time information obtained from cloud providers. This process relies on a tool developed for this purpose [12].

After analyzing the different deployment configurations, the framework automatically selects the one that best satisfies the specified QoS requirements and generates the corresponding deployment artifact in JSON Format ②. From this point, taking into account the information in the JSON file, the DevOps Engineer can manually define the computation distribution scheme using the DSL presented in this work ③, namely *Hybrid Deployments Modeling Language (HDML)* (cf. Section 3.2). One of the purposes of using our HDML is to homogenize the deployment scheme structure allowing its seamless integration with other systems. Besides, in this step the DevOps Engineer adds specific information about the deployment, such as the repository of the source code of each service, the machine information of classical and quantum resources and the mapping of the logical circuit qubits to the physical ones. When a deployment specification is defined with HDML, a model-to-text transformation ④ has been developed to automatically generate

the deployment artifact with YAML notation ⑤, which is needed to distribute the application services across cloud providers.

It is worth explaining that, although standards such as TOSCA² have been proposed to describe the topology and orchestration of cloud-based web services, YAML has become a de facto format for representing configuration and deployment artifacts in many DevOps and cloud-native tools like Docker Compose files³ and Kubernetes manifests⁴ [7]. Therefore, generating a YAML-based artifact facilitates the integration of the proposed process with existing deployment and orchestration ecosystems. In any case, additional M2T transformations targeting different representations can be defined for transforming the HDML specifications.

In summary, our approach provides a declarative way to model hybrid deployments and supports automated model-to-text generation towards target languages, enabling the distribution across quantum and classical computing infrastructures.

3.2 Hybrid Deployments Modeling Language

We have used the Xtext framework for defining a domain-specific language that we call **HDML**. The metamodel corresponding to the grammar defined is depicted in Figure 2. As we can see in the figure legend, the metamodel has been organized into three main groups of classes: (i) classes representing the computing machines (green color), (ii) classes modeling the transpilation process that maps the

²<https://www.oasis-open.org/committees/tosca/faq.php>

³<https://docs.docker.com/reference/compose-file/>

⁴<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

logical qubits of the quantum service onto the physical qubits of the target QPU (orange color), and (iii) classes representing the application services (light blue color). A **Deployment** class acts as root class, containing the following classes:

- **Processing Unit (PU)**: A processing unit is defined by a name, while its specific characteristics depend on the type of computing machine. In the case of a **CPU**, it includes the traditional features associated with classical computing resources. In contrast, a **QPU** includes attributes such as the number of qubits and the quantum technology type, e.g., superconducting or ion-trap. Furthermore, the topology of the quantum computer is represented through the **CouplingMap** class.
- **Qubit**: This class represents the minimum unit of information in quantum computing. The description of qubits used in this work is inspired by the attributes provided by IBM⁵, as this is one of the quantum infrastructure providers that offers the most detailed information. Therefore, the considered attributes are *t1*, *t2*, *prob_meas0_prep1_error*, *prob_meas1_prep0_error*, *readout_error*, and *readout_length*. This class is related to a **QPU** through the **CouplingConnection** class, which represents the connection between two qubits in the topology, and through the **QubitReferenceList** class via the **physicalQubits** relationship, which represents the list of qubits of a machine together with their corresponding attributes. In addition, it is related to the **Service** class through the **Mapping** class, which represents the subset of physical qubits used in the **MappingList** of the quantum service.
- **Service**: This class has a name that identifies it. A service can be either classical (**Classical**) or hybrid (**Hybrid**), depending on whether it corresponds to a traditional service deployed on a classical machine, i.e., a **CPU**, or it is a hybrid service composed of a classical service (**Classical**) and a quantum service (**Quantum**). The quantum service is deployed on a quantum machine, i.e., a **QPU**. Moreover, the logical qubits of the quantum circuit, represented by the **LogicalQubit** class, are mapped onto a subset of the physical qubits of the target QPU. This mapping is represented through the **qubitMapping** relationship.

3.3 Application Scenario

In this section, we use our proposed language, HDML, for defining a deployment instance of the search application scenario described in previous work [12].

The scenario is composed of three *sensor services* that provide sensor data to an *aggregator service*, which is responsible for grouping them and sending them to the *search service*. This search service can be implemented either by a *classical service*, using a binary search algorithm, or by a *hybrid service* using Grover's algorithm for such a search. Finally, a result processing service is responsible for notifying the user if an anomalous value has been detected.

The DevOps Engineer can use our HDML (step ③ in Fig. 1) to specify the deployment displayed in Figure 3 (only an excerpt of the deployment specification is shown for space reasons, the full

```

1 Hybrid "groveralg0"
2 {
3     classical_service:
4         Classical "groveralg00"
5         {
6             repository: "https://github.com/qse-itis
7 -lab/groveralg00"
8             machine: t3large0
9         }
10    quantum_service:
11        Quantum "groveralg0"
12        {
13            machine: ibm_fake_fez
14            Logical_qubits: [q0, q1, q2, q3, q4]
15            qubit_mapping: [
16                (Logical q0, physical_qubits [q1]),
17                (Logical q1, physical_qubits [q5]),
18                (Logical q2, physical_qubits [q4]),
19                (Logical q3, physical_qubits [q3]),
20                (Logical q4, physical_qubits [q0])
21            ]
22        }
23    }

```

Figure 3: Hybrid Service of Search Application

specification is available from our Zenodo repository⁶). The figure shows the fragment corresponding to the deployment description of a hybrid alternative implementing Grover's algorithm. As can be observed, this hybrid service consists of two services. First, a classical service would be deployed on the *t3large* machine provided by AWS, containing the url of the code repository. Second, a quantum service is to be deployed on the IBM provider, specifically on qubits 1, 5, 4, 2, 3 and 0 of the *ibm_fake_fez* machine. Since this specific machine does not correspond to an actual quantum backend, we define a fictitious backend in order to keep the complete example concise and easy to read. Although this is not shown in the example, our HDML also allows specifying a set of physical qubits onto which a logical qubit can be mapped. This may be useful in scenarios where error correction techniques are intended to be applied.

Once the deployment specification is defined, the model-to-text transformation integrated in our framework (step ④ in Fig. 1) takes it as input with the aim of obtaining an artifact that can be interpreted by deployment processes. Specifically, in the current state of our framework, this transformation generates a YAML file (step ⑤ in Fig. 1). A fragment of this file for our application scenario is shown in Figure 4, which serves as input for the subsequent deployment on cloud providers.

4 Related Work

Some approaches have started to apply model-driven techniques to the design of quantum software. This is the case of Pérez-Castillo and Piattini's work [11], where they present a UML profile for the design and analysis of hybrid information systems. This interesting work is closely related to ours. However, their proposal provides a

⁵<https://quantum.cloud.ibm.com/computers>

⁶<https://doi.org/10.5281/zenodo.21135291>



```

1 # This file has been created automatically
2 services:
3 - name: groveralg0
4   type: hybrid
5   classical:
6     name: groveralg00
7     repository: https://github.com/qse-it-is-lab/groveralg00
8     machine:
9       type: CPU
10      instance: t3large0
11   quantum:
12     name: groveralg0
13     machine:
14       type: QPU
15       instance: ibm_fake_fez
16     qubit_mapping:
17       - logical: q0
18         physical_qubits:
19           - name: q1
20       - logical: q1
21         physical_qubits:
22           - name: q5
23       - logical: q2
24         physical_qubits:
25           - name: q4
26       - logical: q3
27         physical_qubits:
28           - name: q3
29       - logical: q4
30         physical_qubits:
31           - name: q0

```

Figure 4: Search Application example of yaml file

UML-based modeling approach for the general design of hybrid systems, while our proposal focuses on a dedicated DSL for specifying deployment aspects in hybrid service-based applications.

Another work, by Gemeinhardt et al. [9], presents a modeling language and design framework for quantum circuits. In this case, they focus on the design of quantum circuits, instead of complete hybrid systems. Our work can be considered complementary in terms of enabling the deployment specification of applications that include such quantum components.

Jiménez-Navajas et al. [10] and Alonso et al. [1] also apply Model-Driven Engineering techniques to quantum software. While the former uses M2T transformations to generate classical and quantum source code from UML models, the latter proposes a model-driven approach for developing quantum programs. Our proposal also uses MDE techniques, but with the aim of obtaining a deployment artifact through M2T transformations.

As can be observed, all these works describe high-level models with the objective of abstracting the design of hybrid systems and quantum software. However, deployment concerns for hybrid classical-quantum service-based applications remain less explored [6]. Unlike previous works, in this paper, a DSL that allows homogenizing the deployment aspects of both classical service-based applications and their hybrid alternatives is proposed. Furthermore, this model is materialized through an M2T transformation into a YAML file that can be used as input for carrying out the deployment of the application.

5 Conclusion and Future Work

In this work, we have proposed the initial definition of a DSL through a grammar that enables the specification, generation, and validation of hybrid deployment description files. The main objective is to provide a specification mechanism that introduces a higher level of abstraction and facilitates the deployment of hybrid quantum-classical service-based applications across different quantum computing providers. In particular, the proposed DSL makes it possible to describe not only the allocation of services to computing resources, but also the mapping between logical and physical qubits within the selected quantum device.

Since this is an initial proposal, it has not yet been validated on physical quantum backends and still includes manual deployment steps. To address these limitations, future work will focus on extending the DSL semantics to support more complex scenarios, such as multiple quantum services sharing a QPU with different physical qubit sets, and by automating the generation of deployment artifacts.

Acknowledgments

This work was partially by FEDER funds under grant PPRO-TIC136-G-2023 and by the Spanish Government (FEDER/Ministerio de Ciencia e Innovación – Agencia Estatal de Investigación) under project IPSCA [PID2021-125527NB-I00].

References

- [1] Diego Alonso, Pedro Sánchez, and Francisco Sánchez-Rubio. 2022. Engineering the development of quantum programs: Application to the Boolean satisfiability problem. *Adv. Eng. Softw.* 173 (2022), 103216. doi:10.1016/J.ADVENGSOFT.2022.103216
- [2] Álvaro M. Aparicio-Morales, José García-Alonso, Javier Cámara, and Juan Manuel Murillo. 2024. Hybrid (Quantum-Classical) Service Applications Deployment Configuration System. In *Service-Oriented Computing - ICSOC 2024*. Springer. doi:10.1007/978-981-96-7423-7_1
- [3] Álvaro M. Aparicio-Morales, Juan Luis Herrera, Enrique Moguel, Javier Berrocal, José García-Alonso, and Juan Manuel Murillo. 2023. Minimizing Deployment Cost of Hybrid Applications. In *Proc. of IEEE International Conference on Quantum Computing (QCE)*. 191–194. doi:10.1109/QCE57702.2023.10209
- [4] Zakaria Abdelmoiz Dahi, Francisco Chicano, and Gabriel Luque. 2024. An Evolutionary Deep Learning Approach for Efficient Quantum Algorithms Transpilation. In *Proc. of 27th European Conference on Applications of Evolutionary Computation*. 240–255. doi:10.1007/978-3-031-56855-8_15
- [5] Jose Garcia-Alonso et al. 2026. Rethinking Services in the Quantum Age: The SOQ Paradigm. *ACM Trans. Softw. Eng. Methodol.* (April 2026). doi:10.1145/3805807 Just Accepted.
- [6] Juan Manuel Murillo et al. 2025. Quantum Software Engineering: Roadmap and Challenges Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 154:1–154:48.
- [7] Yifei Xu et al. 2024. CloudEval-YAML: A Practical Benchmark for Cloud Configuration Generation. In *Proc. of the Seventh Annual Conference on Machine Learning and Systems (MLSys)*.
- [8] José García-Alonso, Javier Rojo, David Valencia, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. 2022. Quantum Software as a Service Through a Quantum API Gateway. *IEEE Internet Comput.* 26, 1 (2022), 34–41.
- [9] Felix Gemeinhardt, Antonio Garmendia, Manuel Wimmer, and Robert Wille. 2024. A Model-Driven Framework for Composition-Based Quantum Circuit Design. *ACM Trans. Quantum Comput.* 5, 4 (2024), 23:1–23:36.
- [10] Luis Jiménez-Navajas, Ricardo Pérez-Castillo, and Mario Piattini. 2025. Code generation for classical-quantum software systems modeled in UML. *Softw. Syst. Model.* 24, 3 (2025), 795–821.
- [11] Ricardo Pérez-Castillo and Mario Piattini. 2022. Design of classical-quantum systems with UML. *Computing* 104, 11 (2022), 2375–2403.
- [12] Álvaro M. Aparicio-Morales, Jose Garcia-Alonso, Javier Cámara, and Juan M. Murillo. 2026. Architecting Hybrid Quantum-Classical Software Systems: Exploration of the Design Trade-off Space with Quantitative Guarantees. arXiv:2606.24260 Submitted to journal.